METHODS

1. Method with Parameter

A method that takes input (parameters) but doesn't necessarily return anything.

Java
```
public class MethodWithParam {      //
Method with a String parameter
public void greetUser(String name) {
        System.out.println("Hello, " + name + "!");
    }
    public static void main(String[] args) {
MethodWithParam obj = new MethodWithParam();
        obj.greetUser("Alice"); // Passing "Alice" as an argument
}
}
```

---

2. Method with Return Type

A method that performs a task and gives back (returns) a value.

Java
```
public class MethodWithReturn {      //
Method with an int return type
public int multiply(int a, int b) {
return a * b;
    }
    public static void main(String[] args) {
        MethodWithReturn obj = new MethodWithReturn();
int result = obj.multiply(5, 4);
        System.out.println("The result is: " + result);
    }
}
```

---

3. Between Classes with a Method as Static

Calling a static method from a different class. You don't need to create an object; you just use the Class Name.

Java
```
class HelperClass {
// Static method
    public static void showMessage() {
        System.out.println("Message from the static method in HelperClass!");
}
}
public class MainClass {
    public static void main(String[] args) {
        // Calling static method using the Class Name directly
        HelperClass.showMessage();
    }
```

```
}
```

---

4. Between Classes with a Method as Static with Parameter

Passing a parameter to a static method in another class.

Java
```
class MathHelper {
    // Static method with a parameter
    public static void printSquare(int number) {
        System.out.println("Square of " + number + " is " + (number * number));
    }
}
public class MainClass {
    public static void main(String[] args) {
        MathHelper.printSquare(6); // Passing 6 as a parameter
}
}
```

---

5. Between Classes with a Method as Static with Return Type

Calling a static method in another class that returns a value.

Java
```
class StringHelper {
    // Static method with a return type
public static String getGreeting() {
return "Welcome to Java Programming!";
    }
}
public class MainClass {
    public static void main(String[] args) {
// Storing the returned value in a variable
        String message = StringHelper.getGreeting();
        System.out.println(message);
    }
}
```

---

6. Non-Static to Static without Parameter and Return Type

Calling a non-static method from a static method (like `main`). **Rule:** You must create an object first.

Java
```
public class NonStaticToStatic {
// Non-static method     public
void display() {
        System.out.println("Non-static method called!");
    }

    // Static method (main)
    public static void main(String[] args) {
```

```
        // Must create an object to call a non-static method from a static context
        NonStaticToStatic obj = new NonStaticToStatic();
obj.display();
    }
}
```

---

7. Non-Static to Static with Method as Parameter

Calling a non-static method that takes a parameter from `main`.

Java

```
public class NonStaticParam {
    // Non-static method with parameter
public void checkAge(int age) {
        System.out.println("Your age is: " + age);
    }      public static void main(String[] args)
{         NonStaticParam obj = new
NonStaticParam();          obj.checkAge(25); //
Passing parameter
    }
}
```

---

8. Non-Static to Static with Method as Return Type

Calling a non-static method that returns a value from `main`.

Java

```
public class NonStaticReturn {
    // Non-static method with return type
public double getPiValue() {
return 3.14159;
    }
    public static void main(String[] args) {
NonStaticReturn obj = new NonStaticReturn();
        double pi = obj.getPiValue(); // Getting the return value
System.out.println("Pi is approximately: " + pi);
    }
}
```

---

9. Between Classes: Non-Static Method Called from a Static Method

Calling a non-static method of *Class A* from the static `main` method of *Class B*.

Java

```
class Worker {
    // Non-static method
public void doWork() {
        System.out.println("Worker is doing the job.");
    }
}
public class Company {
    public static void main(String[] args) {
        // Creating object of another class to access its non-static method
        Worker w1 = new Worker();
w1.doWork();
```

```
    }
}
```

---

10. Non-Static to Static through Reference Variable

This highlights creating a specific named object reference to call the method.

Java

```
public class ReferenceDemo {
public void sayHi() {
        System.out.println("Hi there!");
    }
    public static void main(String[] args) {
        // 'refVar' is the reference variable pointing to the object
ReferenceDemo refVar = new ReferenceDemo();
        refVar.sayHi(); // Using reference variable to call the method
}
}
```

---

11. Pass By Value / Call By Value

When you pass primitive data types (like `int`), a copy is passed. Changing it inside the method does **not** change the original variable. Java

```
public class PassByValue {
    public void changeValue(int number) {
        number = 100; // This only changes the local copy
    }       public static void main(String[]
args) {          PassByValue obj = new
PassByValue();          int myNum = 10;

        System.out.println("Before method: " + myNum); // Prints 10
obj.changeValue(myNum);
        System.out.println("After method: " + myNum);  // Still prints 10
}
}
```

---

12. Pass By Reference / Call By Reference

A quick, candid note: Java is technically strictly "pass-by-value." However, when you pass an object, you are passing the *value of its memory reference*. This means modifying the object's properties inside the method **will** change the original object. Textbooks often refer to this behavior as "Pass by Reference." Java

```
class MyNumber {
int value;
}
public class PassByReference {
    public void changeObjectValue(MyNumber numObj) {
        numObj.value = 100; // Modifying the actual object's property
}
```

```
    public static void main(String[] args) {
        PassByReference test = new PassByReference();
        MyNumber myNum = new MyNumber();
myNum.value = 10;          System.out.println("Before
method: " + myNum.value); // Prints 10
test.changeObjectValue(myNum); // Passing the object
reference          System.out.println("After method:
" + myNum.value);  // Prints 100     }
}
```

---

13. Initialize Variables Through Constructor

A constructor is a special method called automatically when an object is created to set initial values.

Java

```
public class Car {
String color;     int
year;

    // Constructor
public Car() {
color = "Red";
year = 2023;
    }
    public static void main(String[] args) {
        Car myCar = new Car(); // Constructor is called here
        System.out.println("Car Color: " + myCar.color);
        System.out.println("Car Year: " + myCar.year);
    }
}
```

---

14. Initialize Employee ID, Name, Salary Through Constructor

Java

```
public class Employee {
int empId;     String
empName;     double
empSalary;

    // Parameterized Constructor
    public Employee(int id, String name, double salary) {
empId = id;          empName = name;          empSalary =
salary;
    }      public static void main(String[]
args) {          // Initializing during object
creation
        Employee emp1 = new Employee(101, "John Doe", 55000.50);

        System.out.println("ID: " + emp1.empId);
        System.out.println("Name: " + emp1.empName);
        System.out.println("Salary: $" + emp1.empSalary);
    }
}
```

---

15. Initialize Student ID, Name, Fees Through Constructor

Java

```
public class Student {
int studentId;     String
studentName;     double
studentFees;

    // Parameterized Constructor
    public Student(int id, String name, double fees) {
studentId = id;          studentName = name;
studentFees = fees;
    }
    public static void main(String[] args) {
        Student s1 = new Student(5001, "Emma", 1200.00);
        System.out.println("Student ID: " + s1.studentId);
        System.out.println("Student Name: " + s1.studentName);
        System.out.println("Student Fees: $" + s1.studentFees);
}
}
```

---

16. Initialize Employee ID, Name, Salary Using `this` Keyword

When your constructor parameters have the *exact same names* as your class variables, you use the `this` keyword to tell Java which is which. `this.empId` means the class variable, and `empId` means the parameter.

Java

```
public class EmployeeWithThis {
    int empId;
String empName;
double empSalary;

    // Constructor using identical parameter names
    public EmployeeWithThis(int empId, String empName, double empSalary) {
// 'this' refers to the current object's instance variables
this.empId = empId;          this.empName = empName;          this.empSalary
= empSalary;
    }
    public void displayInfo() {
        System.out.println("ID: " + empId + ", Name: " + empName + ", Salary: $" +
empSalary);
    }
    public static void main(String[] args) {
        EmployeeWithThis emp = new EmployeeWithThis(202, "Sarah Smith", 65000.00);
emp.displayInfo();
    } }
```